

Hands-on Exercise: SIMD Intrinsics & Performance Tuning

Parallel & Distributed Computing Lab

What you will build (and why this is fun)

You will implement the same simple math kernel in **7 variants** and watch performance change in real time. The goal is to learn SIMD intrinsics like a scientist: **change one thing, measure, explain.**

We benchmark **AXPY**:

$$y[i] = a \cdot x[i] + y[i]$$

It is intentionally simple so the *performance story* is not buried under complicated logic.

Learning outcomes

By the end, you will be able to:

- Read and write AVX2 intrinsics (256-bit vectors, 8 floats at a time).
- Identify common SIMD issues (tail handling, alignment, aliasing, loop overhead, bandwidth wall).
- Use **perf** counters to distinguish CPU-bound vs memory-bound behavior.
- Scale the optimized SIMD kernel across multiple CPU cores using pthreads.

Exercise rules (important)

- **Do not trust intuition.** Measure everything.
- **Change one thing at a time.** Otherwise you learn nothing.
- **Record results.** Your memory is not a database.

Prerequisites

- Linux machine with AVX2 support
- GCC or Clang, and **perf**

Part 0: Setup

Folder structure (recommended)

Create:

```
mkdir -p simd_workshop && cd simd_workshop
mkdir -p src bin
```

Put the code files from this manual into `src/`.

Perf Permissions (required)

For the exercise (temporary change):

```
sudo sysctl -w kernel.perf_event_paranoid=1
sudo sysctl -w kernel.kptr_restrict=0
```

Note: These are security-sensitive settings; revert after the exercise if needed.

Check CPU SIMD support

```
lscpu | egrep -i "Model name|CPU\ (s\)|Thread|Core"
lscpu | egrep -i "avx2|fma" || echo "WARNING: AVX2/FMA not found!"
```

Compiler flags: one-line explanations

- `-march=native`: use your CPU's supported instructions and tuning.
- `-mavx2`: enable AVX2 vector instructions (256-bit).
- `-mfma`: enable fused multiply-add (one instruction for $a \cdot x + y$).
- `-fno-tree-vectorize`: disable compiler auto-vectorization (keeps baseline scalar honest).
- `-pthread`: enable pthreads and link thread library.
- `-Wall -Wextra` (optional): helpful warnings.
- `-std=c11`: C11 standard, for reproducibility and avoiding compiler level prototype errors.
- `-fno-omit-frame-pointer`: helps profiling stack traces.
- `-O3`: Compiler optimizations

Performance counters we care about

We will focus on:

- `cycles, instructions` $\Rightarrow$ $IPC = \frac{\text{instructions}}{\text{cycles}}$
- `branch-misses` $\Rightarrow$ loop/branch overhead issues
- `cache-misses, LLC-load-misses` $\Rightarrow$ memory pressure / bandwidth

1 Baseline first (Variant 1), “How slow is slow?”

1.1 Build Variant 1 (Scalar baseline)

```
gcc src/v1_axy_scalar.c -fno-tree-vectorize -std=c11 -fno-omit-frame-  
pointer -o bin/v1
```

1.2 Run it

Pick N large enough that data does not fully fit in cache. Default is $N = 16,777,216$ floats ($\approx 64\text{MB}$ per array).

```
./bin/v1 16777216 10 1.2345
```

1.3 Measure with perf

```
perf stat ./bin/v1 16777216 10 1.2345
```

1.4 Things to focus!

1. What is the runtime and bandwidth (GB/s) printed?
2. Compute IPC from perf output: $IPC = \frac{\text{instructions}}{\text{cycles}}$.
3. Is this version doing lots of instructions per element? (Hint: compare later.)

2 Your first SIMD win (Variant 2), “8 floats at once”

2.1 What changes?

We replace scalar operations with AVX2 intrinsics:

- load 8 floats: `_mm256_loadu_ps`
- compute: `_mm256_fmadd_ps` (FMA)
- store: `_mm256_storeu_ps`

2.2 Build Variant 2

```
gcc src/v2_axpy_simd_u.c -march=native -mavx2 -mfma -std=c11 -fno-omit-frame-pointer -o bin/v2
```

2.3 Run and measure

```
./bin/v2 16777216 10 1.2345  
perf stat ./bin/v2 16777216 10 1.2345
```

2.4 What to look for

- **Instructions should drop** vs Variant 1.
- Speedup might be less than $8\times$ because AXPY is often **memory bandwidth limited**.

2.5 Mini-questions

1. What is speedup $S = \frac{T_1}{T_2}$?
2. Did IPC increase or decrease? Why might memory-bound code have weird IPC behavior?

Library reference (Variant 2B), “How good can optimized code get?”

Why compare against OpenBLAS?

Before further hand-tuning, we compare our SIMD implementation against a **highly optimized, production-grade library**.

OpenBLAS provides hand-tuned kernels (often written in assembly) with:

- aggressive vectorization,
- careful unrolling and scheduling,
- architecture-specific tuning.

This variant answers a critical question:

Are we anywhere close to what experts achieve?

Important: single-thread only

To keep the comparison fair:

- OpenBLAS is forced to use **one thread**.
- We compare against Variant 2 (single-thread SIMD).

Build Variant 2B (OpenBLAS)

```
gcc src/v2_openblas_saxpy.c -O3 -march=native -std=c11 \
    -lopenblas -fno-omit-frame-pointer -o bin/v2_oBLAS
```

Run and measure

```
export OPENBLAS_NUM_THREADS=1
export OMP_NUM_THREADS=1

./bin/v2_oBLAS 16777216 10 1.2345
perf stat ./bin/v2_oBLAS 16777216 10 1.2345
```

What to look for

- How close is OpenBLAS to Variant 2?
- Does it execute fewer instructions?
- Does it reach higher effective bandwidth?

Key insight

If OpenBLAS outperforms our SIMD code, it does *not* mean our code is bad. It means:

- memory behavior,
- instruction scheduling,
- and store policies

still matter even after basic SIMD vectorization.

3 The alignment myth (Variant 3, “Sometimes it matters, sometimes it doesn’t”)

3.1 Issue: alignment

Aligned loads/stores (`load_ps/store_ps`) assume 32-byte alignment. If misaligned, things can break or slow down.

Aligned accesses reduce the chance that a single SIMD load spans multiple cache lines. While modern CPUs handle unaligned loads efficiently, alignment can still:

- reduce micro-operations,
- improve cache-line utilization,
- and simplify the memory access pattern.

This optimization represents **memory-access hygiene**: correct, clean, and occasionally beneficial but rarely transformative on its own.

3.2 Fix

We allocate aligned arrays and use aligned loads/stores. This is a great example of an optimization that is **good hygiene** but not always a big win.

3.3 Build and run Variant 3

```
gcc src/v3_axpy_simd_aligned.c -march=native -mavx2 -mfma -std=c11 -fno
  -omit-frame-pointer -o bin/v3

./bin/v3 16777216 10 1.2345
perf stat ./bin/v3 16777216 10 1.2345
```

3.4 Mini-questions

1. Compare Variant 2 vs 3. Did alignment help? How much?
2. If it barely helped, what does that imply about modern CPUs’ unaligned handling?

4 Correctness + compiler freedom (Variant 4), “Tail + restrict”

4.1 Issue A: tail elements

SIMD processes 8 floats per iteration. If N is not divisible by 8, the remainder needs handling.

Why this optimization exists The `restrict` qualifier is a **promise to the compiler** that memory regions do not overlap. This removes aliasing ambiguity and allows more aggressive reordering of loads and stores.

Explicit tail handling ensures that:

- the SIMD loop has no bounds checks,
- correctness is preserved for arbitrary N ,
- and the hot loop remains simple and predictable.

This variant improves *compiler freedom and correctness*, not raw throughput.

4.2 Issue B: aliasing

Without `restrict`, the compiler must assume pointers might overlap and becomes conservative.

4.3 Fix

Variant 4 uses:

- explicit tail loop
- `restrict` pointers

4.4 Build and run Variant 4

```
gcc src/v4_axpy_simd_restrict_tail.c -march=native -mavx2 -mfma -std=c11 -fno-omit-frame-pointer -o bin/v4

./bin/v4 16777216 10 1.2345
perf stat ./bin/v4 16777216 10 1.2345
```

4.5 Mini-questions

1. Did `restrict` help? Why would it help even if we already used intrinsics?
2. What happens if you run with an N that is not divisible by 8? (Try $N = 16777219$.)

5 Reduce overhead, increase ILP (Variant 5), “Unroll like you mean it”

5.1 Issue: loop overhead and latency hiding

Even with SIMD, the loop can be limited by:

- branch + loop bookkeeping
- not enough independent work to hide memory latency

Why unrolling can help Loop unrolling:

- reduces branch and loop-counter overhead,
- exposes more independent instructions,
- increases instruction-level parallelism (ILP),
- allows more memory operations to be in flight.

Unrolling is most effective when latency not bandwidth, is the limiting factor. For large streaming workloads, benefits may be small once DRAM bandwidth saturates.

5.2 Fix: unroll by 4 vectors

We process 32 floats per iteration (4×8 -float vectors). This reduces loop overhead and increases instruction-level parallelism.

5.3 Build and run Variant 5

```
gcc src/v5_axpy_simd_unroll4.c -march=native -mavx2 -mfma -std=c11 -fno-omit-frame-pointer -o bin/v5

./bin/v5 16777216 10 1.2345
perf stat ./bin/v5 16777216 10 1.2345
```

5.4 Mini-questions

1. Did branch misses drop compared to Variant 4?
2. Did performance improve? If not much, what does that suggest (hint: bandwidth wall)?

6 Prefetch experiment (Variant 6), “Optimizations are hypotheses”

6.1 Issue: prefetch is situational

Modern CPUs already prefetch sequential streams well. Manual prefetch can:

- help slightly,
- do nothing,
- or hurt by polluting caches.

Why this is an experiment Modern CPUs already include sophisticated hardware prefetchers for linear access patterns. Manual prefetching:

- may help for irregular or sparse access,
- may be redundant for streaming arrays,
- may even hurt by increasing instruction overhead or cache pollution.

This variant demonstrates that not all optimizations are guaranteed wins; some are **hypotheses that must be validated experimentally**.

6.2 Build and run Variant 6

```
gcc src/v6_axpy_simd_unroll4_prefetch.c -march=native -mavx2 -mfma -  
std=c11 -fno-omit-frame-pointer -o bin/v6  
  
./bin/v6 16777216 10 1.2345  
perf stat ./bin/v6 16777216 10 1.2345
```

6.3 Mini-questions

1. Did it improve? If yes, how much? If no, why might hardware prefetch already be enough?
2. What could you tune? (Hint: prefetch distance.)

7 Multi-core + SIMD (Variant 7), “Now you meet the bandwidth boss”

7.1 Issue: scaling hits memory bandwidth

AXPY moves lots of bytes: read x (4B), read y (4B), write y (4B) $\Rightarrow$ about 12 bytes/element. At some thread count, DRAM bandwidth saturates, and speedup flattens.

7.2 Fix

We split the array into chunks and run the best SIMD kernel per thread.

7.3 Build Variant 7

```
gcc src/v7_axpy_pthreads_simd.c -march=native -mavx2 -mfma -std=c11 -  
fno-omit-frame-pointer -pthread -o bin/v7
```

7.4 Run thread scaling experiment

```
# Args: N R a T pin  
./bin/v7 16777216 10 1.2345 1 0  
./bin/v7 16777216 10 1.2345 2 0  
./bin/v7 16777216 10 1.2345 4 0  
./bin/v7 16777216 10 1.2345 8 0  
  
# With pinning (Linux): pin=1 will try to bind thread t to CPU t  
./bin/v7 16777216 10 1.2345 8 1
```

Measure with perf:

```
perf stat ./bin/v7 16777216 10 1.2345 8 1
```

7.5 Mini-questions

1. Plot (mentally or in a table) speedup vs threads. Where does it flatten?
2. Why does pinning sometimes help? (Hint: thread migration and cache locality.)

Results analysis and results table template

You can fill the following table for a comparison between all the variants.

Instructions

For each variant, record:

- runtime (sec) from program output
- estimated bandwidth (GB/s) from program output
- cycles, instructions, cache-misses, LLC-load-misses from perf
- compute IPC and speedup vs Variant 1

Variant	Time (s)	BW (GB/s)	Speedup	Instructions	Cycles	IPC
V1 Scalar			1.00			
V2 SIMD unaligned						
V2 OpenBlas Variant						
V3 SIMD aligned						
V4 SIMD+restrict+tail						
V5 SIMD+unroll4						
V6 SIMD+unroll4+prefetch						
V7 Pthreads+SIMD (T=...)						

Table 1: Performance summary. Compute $IPC = \frac{\text{instructions}}{\text{cycles}}$ and Speedup = $\frac{T_{V1}}{T_{Vx}}$.

Short analysis prompts

Think of these like a performance engineer:

1. Which variant gave the **first big improvement** and why?
2. Which optimization gave the **least benefit** and what does that imply?
3. At what thread count does scaling flatten in Variant 7? Why?
4. Based on cache/LLC misses, is the kernel compute-bound or memory-bound?

Appendix A: Command Line Arguments

7.6 Full parameter meaning (v1–v7)

All programs follow a consistent command-line pattern. The first three arguments are common across all variants (v1–v7). Variant v7 adds two more for threading.

Argument	Name	Meaning (one line)
1	N	Number of elements in arrays x and y (problem size).
2	R	Repeat count: how many timed iterations of the kernel to run.
3	a	Scalar multiplier in $y[i] = a \cdot x[i] + y[i]$.
4 (v7 only)	T	Number of pthreads (threads) to use.
5 (v7 only)	<code>pin</code>	Thread pinning flag: 1 pins thread t to CPU t (Linux), 0 disables.

Table 2: Command-line parameters for variants v1–v7.

Command formats

- **Variants v1–v6:**

`./bin/vX` N R a

Example:

```
./bin/v1 16777216 10 1.2345
```

- **Variant v7 (Pthreads + SIMD):**

`./bin/v7` N R a T `pin`

Example:

```
./bin/v7 16777216 10 1.2345 8 1
```

Why R exists (measurement stability) R is not part of the algorithm; it is a measurement tool. Repeating the kernel makes the timed region long enough to reduce noise from timer resolution, OS scheduling jitter, and one-time effects. For fair comparisons, use the **same** N and R across all variants.

Notes on v7-specific parameters

- T controls parallelism across CPU cores. Increasing T helps until memory bandwidth saturates.
- `pin=1` or Affinity, this may improve consistency by reducing thread migration; it can also have no effect depending on the system.

Appendix B: Code files (copy into src/)

The code files are also available in .ZIP archive along with this Exercise on GCR. The following code listings correspond to:

- src/v1_axpy_scalar.c
- src/v2_axpy_simd_u.c
- src/v2_openblas_saxpy.c
- src/v3_axpy_simd_aligned.c
- src/v4_axpy_simd_restrict_tail.c
- src/v5_axpy_simd_unroll4.c
- src/v6_axpy_simd_unroll4_prefetch.c
- src/v7_axpy_pthreads_simd.c

V1: Scalar baseline

```
1 // v1_axpy_scalar.c
2 #define _GNU_SOURCE
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <stdint.h>
6 #include <time.h>
7
8 static inline double now_sec(void) {
9     struct timespec ts;
10    clock_gettime(CLOCK_MONOTONIC, &ts);
11    return (double)ts.tv_sec + 1e-9 * (double)ts.tv_nsec;
12 }
13
14 static void init_arrays(float *x, float *y, size_t N) {
15     for (size_t i = 0; i < N; i++) {
16         x[i] = (float)(0.001 * (double)(i % 1000));
17         y[i] = (float)(0.002 * (double)((i + 7) % 1000));
18     }
19 }
20
21 static float checksum(const float *y, size_t N) {
22     double s = 0.0;
23     for (size_t i = 0; i < N; i += (N/16 + 1)) s += y[i];
24     return (float)s;
25 }
26
27 static void axpy_scalar(float *y, const float *x, float a, size_t N) {
28     for (size_t i = 0; i < N; i++) {
29         y[i] = a * x[i] + y[i];
30     }
31 }
32
33 int main(int argc, char **argv) {
34     size_t N = (argc > 1) ? (size_t)atoll(argv[1]) : (size_t)16 * 1024
35         * 1024;
36     int R = (argc > 2) ? atoi(argv[2]) : 10;
```

```

36     float a = (argc > 3) ? (float)atof(argv[3]) : 1.2345f;
37
38     float *x = (float*)malloc(N * sizeof(float));
39     float *y = (float*)malloc(N * sizeof(float));
40     if (!x || !y) { fprintf(stderr, "alloc failed\n"); return 1; }
41
42     init_arrays(x, y, N);
43
44     // warmup
45     axpy_scalar(y, x, a, N);
46
47     double t0 = now_sec();
48     for (int r = 0; r < R; r++) axpy_scalar(y, x, a, N);
49     double t1 = now_sec();
50
51     double sec = t1 - t0;
52     double bytes = 12.0 * (double)N * (double)R; // ~12 bytes per
53         element
54     double gbps = (bytes / sec) / 1e9;
55
56     printf("Variant 1 (Scalar)  N=%zu R=%d  time=%.6f s  BW~%.2f GB/s
57         checksum=%f\n",
58         N, R, sec, gbps, checksum(y, N));
59
60     free(x); free(y);
61     return 0;
62 }

```

V2: SIMD (unaligned load/store)

```

1  // v2_axpy_simd_u.c
2  #define _GNU_SOURCE
3  #include <stdio.h>
4  #include <stdlib.h>
5  #include <stdint.h>
6  #include <time.h>
7  #include <immintrin.h>
8
9  static inline double now_sec(void) {
10     struct timespec ts;
11     clock_gettime(CLOCK_MONOTONIC, &ts);
12     return (double)ts.tv_sec + 1e-9 * (double)ts.tv_nsec;
13 }
14
15 static void init_arrays(float *x, float *y, size_t N) {
16     for (size_t i = 0; i < N; i++) {
17         x[i] = (float)(0.001 * (double)(i % 1000));
18         y[i] = (float)(0.002 * (double)((i + 7) % 1000));
19     }
20 }
21
22 static float checksum(const float *y, size_t N) {
23     double s = 0.0;
24     for (size_t i = 0; i < N; i += (N/16 + 1)) s += y[i];
25     return (float)s;
26 }
27

```

```

28 static void axpy_simd_u(float *y, const float *x, float a, size_t N) {
29     __m256 va = _mm256_set1_ps(a);
30     size_t i = 0;
31     size_t limit = N & ~(size_t)7; // floor to multiple of 8
32     for (; i < limit; i += 8) {
33         __m256 vx = _mm256_loadu_ps(x + i);
34         __m256 vy = _mm256_loadu_ps(y + i);
35         __m256 vout = _mm256_fmadd_ps(va, vx, vy); // a*x + y
36         _mm256_storeu_ps(y + i, vout);
37     }
38     for (; i < N; i++) y[i] = a * x[i] + y[i]; // tail
39 }
40
41 int main(int argc, char **argv) {
42     size_t N = (argc > 1) ? (size_t)atoll(argv[1]) : (size_t)16 * 1024
        * 1024;
43     int R = (argc > 2) ? atoi(argv[2]) : 10;
44     float a = (argc > 3) ? (float)atof(argv[3]) : 1.2345f;
45
46     float *x = (float*)malloc(N * sizeof(float));
47     float *y = (float*)malloc(N * sizeof(float));
48     if (!x || !y) { fprintf(stderr, "alloc failed\n"); return 1; }
49
50     init_arrays(x, y, N);
51     axpy_simd_u(y, x, a, N); // warmup
52
53     double t0 = now_sec();
54     for (int r = 0; r < R; r++) axpy_simd_u(y, x, a, N);
55     double t1 = now_sec();
56
57     double sec = t1 - t0;
58     double bytes = 12.0 * (double)N * (double)R;
59     double gbps = (bytes / sec) / 1e9;
60
61     printf("Variant 2 (SIMD unaligned)  N=%zu R=%d  time=%.6f s  BW~%.2
        f GB/s  checksum=%f\n",
62           N, R, sec, gbps, checksum(y, N));
63
64     free(x); free(y);
65     return 0;
66 }

```

V2: SIMD (openBLAS saxpy())

```

1 // v2_openblas_saxpy.c
2 #define _GNU_SOURCE
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <stdint.h>
6 #include <time.h>
7 #include <blas.h>
8
9 #ifdef HAVE_OPENBLAS_SET_THREADS
10 #include <openblas_config.h>
11 #endif
12
13 static inline double now_sec(void) {

```

```

14     struct timespec ts;
15     clock_gettime(CLOCK_MONOTONIC, &ts);
16     return (double)ts.tv_sec + 1e-9 * (double)ts.tv_nsec;
17 }
18
19 static void init_arrays(float *x, float *y, size_t N) {
20     for (size_t i = 0; i < N; i++) {
21         x[i] = (float)(0.001 * (double)(i % 1000));
22         y[i] = (float)(0.002 * (double)((i + 7) % 1000));
23     }
24 }
25
26 static float checksum(const float *y, size_t N) {
27     double s = 0.0;
28     for (size_t i = 0; i < N; i += (N/16 + 1)) s += y[i];
29     return (float)s;
30 }
31
32 int main(int argc, char **argv) {
33     size_t N = (argc > 1) ? (size_t)atoll(argv[1]) : (size_t)16 * 1024
        * 1024;
34     int R = (argc > 2) ? atoi(argv[2]) : 10;
35     float a = (argc > 3) ? (float)atof(argv[3]) : 1.2345f;
36
37     float *x = (float*)malloc(N * sizeof(float));
38     float *y = (float*)malloc(N * sizeof(float));
39     if (!x || !y) { fprintf(stderr, "alloc failed\n"); return 1; }
40
41     init_arrays(x, y, N);
42
43     // Force single-thread if supported at compile-time (optional)
44 #ifdef HAVE_OPENBLAS_SET_THREADS
45     openblas_set_num_threads(1);
46 #endif
47
48     // warmup
49     cblas_saxpy((int)N, a, x, 1, y, 1);
50
51     double t0 = now_sec();
52     for (int r = 0; r < R; r++) {
53         cblas_saxpy((int)N, a, x, 1, y, 1);
54     }
55     double t1 = now_sec();
56
57     double sec = t1 - t0;
58     double bytes = 12.0 * (double)N * (double)R; // ~12 bytes/element
59     double gbps = (bytes / sec) / 1e9;
60
61     printf("OpenBLAS saxpy (1 thread) N=%zu R=%d time=%.6f s BW~%.2f
        GB/s checksum=%f\n",
62         N, R, sec, gbps, checksum(y, N));
63
64     free(x); free(y);
65     return 0;
66 }

```


V3: SIMD (aligned alloc + aligned load/store)

```
1 // v3_axpy_simd_aligned.c
2 #define _GNU_SOURCE
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <stdint.h>
6 #include <time.h>
7 #include <immintrin.h>
8
9 static inline double now_sec(void) {
10     struct timespec ts;
11     clock_gettime(CLOCK_MONOTONIC, &ts);
12     return (double)ts.tv_sec + 1e-9 * (double)ts.tv_nsec;
13 }
14
15 static void init_arrays(float *x, float *y, size_t N) {
16     for (size_t i = 0; i < N; i++) {
17         x[i] = (float)(0.001 * (double)(i % 1000));
18         y[i] = (float)(0.002 * (double)((i + 7) % 1000));
19     }
20 }
21
22 static float checksum(const float *y, size_t N) {
23     double s = 0.0;
24     for (size_t i = 0; i < N; i += (N/16 + 1)) s += y[i];
25     return (float)s;
26 }
27
28 static void axpy_simd_aligned(float *y, const float *x, float a, size_t
    N) {
29     __m256 va = _mm256_set1_ps(a);
30     size_t i = 0;
31     size_t limit = N & ~(size_t)7;
32     for (; i < limit; i += 8) {
33         __m256 vx = _mm256_load_ps(x + i);
34         __m256 vy = _mm256_load_ps(y + i);
35         __m256 vout = _mm256_fmadd_ps(va, vx, vy);
36         _mm256_store_ps(y + i, vout);
37     }
38     for (; i < N; i++) y[i] = a * x[i] + y[i];
39 }
40
41 int main(int argc, char **argv) {
42     size_t N = (argc > 1) ? (size_t)atoll(argv[1]) : (size_t)16 * 1024
        * 1024;
43     int R = (argc > 2) ? atoi(argv[2]) : 10;
44     float a = (argc > 3) ? (float)atof(argv[3]) : 1.2345f;
45
46     float *x = NULL, *y = NULL;
47     if (posix_memalign((void**)&x, 32, N * sizeof(float)) != 0) return
        1;
48     if (posix_memalign((void**)&y, 32, N * sizeof(float)) != 0) return
        1;
49
50     init_arrays(x, y, N);
51     axpy_simd_aligned(y, x, a, N); // warmup
52 }
```

```

53     double t0 = now_sec();
54     for (int r = 0; r < R; r++) axpy_simd_aligned(y, x, a, N);
55     double t1 = now_sec();
56
57     double sec = t1 - t0;
58     double bytes = 12.0 * (double)N * (double)R;
59     double gbps = (bytes / sec) / 1e9;
60
61     printf("Variant 3 (SIMD aligned)  N=%zu R=%d  time=%.6f s  BW~%.2f
62           GB/s  checksum=%f\n",
63           N, R, sec, gbps, checksum(y, N));
64
65     free(x); free(y);
66     return 0;
67 }

```

V4: SIMD + restrict + tail handling

```

1  // v4_axpy_simd_restrict_tail.c
2  #define _GNU_SOURCE
3  #include <stdio.h>
4  #include <stdlib.h>
5  #include <stdint.h>
6  #include <time.h>
7  #include <immintrin.h>
8
9  static inline double now_sec(void) {
10     struct timespec ts;
11     clock_gettime(CLOCK_MONOTONIC, &ts);
12     return (double)ts.tv_sec + 1e-9 * (double)ts.tv_nsec;
13 }
14
15 static void init_arrays(float *x, float *y, size_t N) {
16     for (size_t i = 0; i < N; i++) {
17         x[i] = (float)(0.001 * (double)(i % 1000));
18         y[i] = (float)(0.002 * (double)((i + 7) % 1000));
19     }
20 }
21
22 static float checksum(const float *y, size_t N) {
23     double s = 0.0;
24     for (size_t i = 0; i < N; i += (N/16 + 1)) s += y[i];
25     return (float)s;
26 }
27
28 static void axpy_simd_restrict(float * restrict y,
29                                const float * restrict x,
30                                float a, size_t N)
31 {
32     __m256 va = _mm256_set1_ps(a);
33     size_t limit = N & ~(size_t)7;
34
35     size_t i = 0;
36     for (; i < limit; i += 8) {
37         __m256 vx = _mm256_loadu_ps(x + i);
38         __m256 vy = _mm256_loadu_ps(y + i);
39         __m256 vout = _mm256_fmadd_ps(va, vx, vy);

```

```

40     _mm256_storeu_ps(y + i, vout);
41 }
42 for (; i < N; i++) y[i] = a * x[i] + y[i];
43 }
44
45 int main(int argc, char **argv) {
46     size_t N = (argc > 1) ? (size_t)atoll(argv[1]) : (size_t)16 * 1024
        * 1024;
47     int R = (argc > 2) ? atoi(argv[2]) : 10;
48     float a = (argc > 3) ? (float)atof(argv[3]) : 1.2345f;
49
50     float *x = (float*)malloc(N * sizeof(float));
51     float *y = (float*)malloc(N * sizeof(float));
52     if (!x || !y) { fprintf(stderr, "alloc failed\n"); return 1; }
53
54     init_arrays(x, y, N);
55     axpy_simd_restrict(y, x, a, N); // warmup
56
57     double t0 = now_sec();
58     for (int r = 0; r < R; r++) axpy_simd_restrict(y, x, a, N);
59     double t1 = now_sec();
60
61     double sec = t1 - t0;
62     double bytes = 12.0 * (double)N * (double)R;
63     double gbps = (bytes / sec) / 1e9;
64
65     printf("Variant 4 (SIMD + restrict + tail)  N=%zu R=%d  time=%.6f s
        BW~%.2f GB/s  checksum=%f\n",
66           N, R, sec, gbps, checksum(y, N));
67
68     free(x); free(y);
69     return 0;
70 }

```

V5: SIMD + unroll4

```

1  // v5_axpy_simd_unroll4.c
2  #define _GNU_SOURCE
3  #include <stdio.h>
4  #include <stdlib.h>
5  #include <stdint.h>
6  #include <time.h>
7  #include <immintrin.h>
8
9  static inline double now_sec(void) {
10     struct timespec ts;
11     clock_gettime(CLOCK_MONOTONIC, &ts);
12     return (double)ts.tv_sec + 1e-9 * (double)ts.tv_nsec;
13 }
14
15 static void init_arrays(float *x, float *y, size_t N) {
16     for (size_t i = 0; i < N; i++) {
17         x[i] = (float)(0.001 * (double)(i % 1000));
18         y[i] = (float)(0.002 * (double)((i + 7) % 1000));
19     }
20 }
21

```

```

22 static float checksum(const float *y, size_t N) {
23     double s = 0.0;
24     for (size_t i = 0; i < N; i += (N/16 + 1)) s += y[i];
25     return (float)s;
26 }
27
28 static void axpy_simd_unroll4(float * restrict y,
29                               const float * restrict x,
30                               float a, size_t N)
31 {
32     __m256 va = _mm256_set1_ps(a);
33     size_t i = 0;
34     size_t limit = N & ~(size_t)31; // multiple of 32 floats
35
36     for (; i < limit; i += 32) {
37         __m256 x0 = _mm256_loadu_ps(x + i + 0);
38         __m256 y0 = _mm256_loadu_ps(y + i + 0);
39         __m256 x1 = _mm256_loadu_ps(x + i + 8);
40         __m256 y1 = _mm256_loadu_ps(y + i + 8);
41         __m256 x2 = _mm256_loadu_ps(x + i + 16);
42         __m256 y2 = _mm256_loadu_ps(y + i + 16);
43         __m256 x3 = _mm256_loadu_ps(x + i + 24);
44         __m256 y3 = _mm256_loadu_ps(y + i + 24);
45
46         y0 = _mm256_fmadd_ps(va, x0, y0);
47         y1 = _mm256_fmadd_ps(va, x1, y1);
48         y2 = _mm256_fmadd_ps(va, x2, y2);
49         y3 = _mm256_fmadd_ps(va, x3, y3);
50
51         _mm256_storeu_ps(y + i + 0, y0);
52         _mm256_storeu_ps(y + i + 8, y1);
53         _mm256_storeu_ps(y + i + 16, y2);
54         _mm256_storeu_ps(y + i + 24, y3);
55     }
56
57     // leftover: do 8-wide then scalar tail
58     size_t limit8 = N & ~(size_t)7;
59     for (; i < limit8; i += 8) {
60         __m256 vx = _mm256_loadu_ps(x + i);
61         __m256 vy = _mm256_loadu_ps(y + i);
62         __m256 vout = _mm256_fmadd_ps(va, vx, vy);
63         _mm256_storeu_ps(y + i, vout);
64     }
65     for (; i < N; i++) y[i] = a * x[i] + y[i];
66 }
67
68 int main(int argc, char **argv) {
69     size_t N = (argc > 1) ? (size_t)atoll(argv[1]) : (size_t)16 * 1024
70     * 1024;
71     int R = (argc > 2) ? atoi(argv[2]) : 10;
72     float a = (argc > 3) ? (float)atof(argv[3]) : 1.2345f;
73
74     float *x = (float*)malloc(N * sizeof(float));
75     float *y = (float*)malloc(N * sizeof(float));
76     if (!x || !y) { fprintf(stderr, "alloc failed\n"); return 1; }
77
78     init_arrays(x, y, N);
79     axpy_simd_unroll4(y, x, a, N); // warmup

```

```

79
80     double t0 = now_sec();
81     for (int r = 0; r < R; r++) axpy_simd_unroll4(y, x, a, N);
82     double t1 = now_sec();
83
84     double sec = t1 - t0;
85     double bytes = 12.0 * (double)N * (double)R;
86     double gbps = (bytes / sec) / 1e9;
87
88     printf("Variant 5 (SIMD + unroll4)  N=%zu R=%d  time=%.6f s  BW~%.2
89           f GB/s  checksum=%f\n",
90           N, R, sec, gbps, checksum(y, N));
91
92     free(x); free(y);
93     return 0;
94 }

```

V6: SIMD + unroll4 + prefetch experiment

```

1  // v6_axpy_simd_unroll4_prefetch.c
2  #define _GNU_SOURCE
3  #include <stdio.h>
4  #include <stdlib.h>
5  #include <stdint.h>
6  #include <time.h>
7  #include <immintrin.h>
8
9  static inline double now_sec(void) {
10     struct timespec ts;
11     clock_gettime(CLOCK_MONOTONIC, &ts);
12     return (double)ts.tv_sec + 1e-9 * (double)ts.tv_nsec;
13 }
14
15 static void init_arrays(float *x, float *y, size_t N) {
16     for (size_t i = 0; i < N; i++) {
17         x[i] = (float)(0.001 * (double)(i % 1000));
18         y[i] = (float)(0.002 * (double)((i + 7) % 1000));
19     }
20 }
21
22 static float checksum(const float *y, size_t N) {
23     double s = 0.0;
24     for (size_t i = 0; i < N; i += (N/16 + 1)) s += y[i];
25     return (float)s;
26 }
27
28 static void axpy_simd_unroll4_prefetch(float * restrict y,
29                                         const float * restrict x,
30                                         float a, size_t N)
31 {
32     __m256 va = _mm256_set1_ps(a);
33     size_t i = 0;
34     size_t limit = N & ~(size_t)31;
35
36     const size_t PF_DIST = 256; // floats ahead (tunable)
37     for (; i < limit; i += 32) {
38         _mm_prefetch((const char*)(x + i + PF_DIST), _MM_HINT_T0);

```

```

39     _mm_prefetch((const char*)(y + i + PF_DIST), _MM_HINT_T0);
40
41     __m256 x0 = _mm256_loadu_ps(x + i + 0);
42     __m256 y0 = _mm256_loadu_ps(y + i + 0);
43     __m256 x1 = _mm256_loadu_ps(x + i + 8);
44     __m256 y1 = _mm256_loadu_ps(y + i + 8);
45     __m256 x2 = _mm256_loadu_ps(x + i + 16);
46     __m256 y2 = _mm256_loadu_ps(y + i + 16);
47     __m256 x3 = _mm256_loadu_ps(x + i + 24);
48     __m256 y3 = _mm256_loadu_ps(y + i + 24);
49
50     y0 = _mm256_fmadd_ps(va, x0, y0);
51     y1 = _mm256_fmadd_ps(va, x1, y1);
52     y2 = _mm256_fmadd_ps(va, x2, y2);
53     y3 = _mm256_fmadd_ps(va, x3, y3);
54
55     _mm256_storeu_ps(y + i + 0, y0);
56     _mm256_storeu_ps(y + i + 8, y1);
57     _mm256_storeu_ps(y + i + 16, y2);
58     _mm256_storeu_ps(y + i + 24, y3);
59 }
60
61 size_t limit8 = N & ~(size_t)7;
62 for (; i < limit8; i += 8) {
63     __m256 vx = _mm256_loadu_ps(x + i);
64     __m256 vy = _mm256_loadu_ps(y + i);
65     __m256 vout = _mm256_fmadd_ps(va, vx, vy);
66     _mm256_storeu_ps(y + i, vout);
67 }
68 for (; i < N; i++) y[i] = a * x[i] + y[i];
69 }
70
71 int main(int argc, char **argv) {
72     size_t N = (argc > 1) ? (size_t)atoll(argv[1]) : (size_t)16 * 1024
73     * 1024;
74     int R = (argc > 2) ? atoi(argv[2]) : 10;
75     float a = (argc > 3) ? (float)atof(argv[3]) : 1.2345f;
76
77     float *x = (float*)malloc(N * sizeof(float));
78     float *y = (float*)malloc(N * sizeof(float));
79     if (!x || !y) { fprintf(stderr, "alloc failed\n"); return 1; }
80
81     init_arrays(x, y, N);
82     axpy_simd_unroll4_prefetch(y, x, a, N); // warmup
83
84     double t0 = now_sec();
85     for (int r = 0; r < R; r++) axpy_simd_unroll4_prefetch(y, x, a, N);
86     double t1 = now_sec();
87
88     double sec = t1 - t0;
89     double bytes = 12.0 * (double)N * (double)R;
90     double gbps = (bytes / sec) / 1e9;
91
92     printf("Variant 6 (SIMD + unroll4 + prefetch)  N=%zu R=%d  time=%.6
93     f s  BW~%.2f GB/s  checksum=%f\n",
94     N, R, sec, gbps, checksum(y, N));
95
96     free(x); free(y);

```

```

95     return 0;
96 }

```

V7: Pthreads + SIMD

```

1 // v7_axpy_pthreads_simd.c
2 #define _GNU_SOURCE
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <stdint.h>
6 #include <time.h>
7 #include <pthread.h>
8 #include <immintrin.h>
9 #ifdef __linux__
10 #include <sched.h>
11 #endif
12
13 static inline double now_sec(void) {
14     struct timespec ts;
15     clock_gettime(CLOCK_MONOTONIC, &ts);
16     return (double)ts.tv_sec + 1e-9 * (double)ts.tv_nsec;
17 }
18
19 static void init_arrays(float *x, float *y, size_t N) {
20     for (size_t i = 0; i < N; i++) {
21         x[i] = (float)(0.001 * (double)(i % 1000));
22         y[i] = (float)(0.002 * (double)((i + 7) % 1000));
23     }
24 }
25
26 static float checksum(const float *y, size_t N) {
27     double s = 0.0;
28     for (size_t i = 0; i < N; i += (N/16 + 1)) s += y[i];
29     return (float)s;
30 }
31
32 // Best kernel: Variant 5 style (unroll4)
33 static void axpy_simd_unroll4_chunk(float * restrict y,
34                                     const float * restrict x,
35                                     float a, size_t begin, size_t end)
36 {
37     __m256 va = _mm256_set1_ps(a);
38
39     size_t i = begin;
40     size_t len = end - begin;
41
42     // Make sure we don't run past end; unrolled loop needs 32-float
43     // blocks
44     size_t limit = begin + (len & ~(size_t)31);
45
46     for (; i < limit; i += 32) {
47         __m256 x0 = _mm256_loadu_ps(x + i + 0);
48         __m256 y0 = _mm256_loadu_ps(y + i + 0);
49         __m256 x1 = _mm256_loadu_ps(x + i + 8);
50         __m256 y1 = _mm256_loadu_ps(y + i + 8);
51         __m256 x2 = _mm256_loadu_ps(x + i + 16);
52         __m256 y2 = _mm256_loadu_ps(y + i + 16);

```

```

52     __m256 x3 = _mm256_loadu_ps(x + i + 24);
53     __m256 y3 = _mm256_loadu_ps(y + i + 24);
54
55     y0 = _mm256_fmadd_ps(va, x0, y0);
56     y1 = _mm256_fmadd_ps(va, x1, y1);
57     y2 = _mm256_fmadd_ps(va, x2, y2);
58     y3 = _mm256_fmadd_ps(va, x3, y3);
59
60     _mm256_storeu_ps(y + i + 0, y0);
61     _mm256_storeu_ps(y + i + 8, y1);
62     _mm256_storeu_ps(y + i + 16, y2);
63     _mm256_storeu_ps(y + i + 24, y3);
64 }
65
66 size_t limit8 = begin + (len & ~(size_t)7);
67 for (; i < limit8; i += 8) {
68     __m256 vx = _mm256_loadu_ps(x + i);
69     __m256 vy = _mm256_loadu_ps(y + i);
70     __m256 vout = _mm256_fmadd_ps(va, vx, vy);
71     _mm256_storeu_ps(y + i, vout);
72 }
73 for (; i < end; i++) y[i] = a * x[i] + y[i];
74 }
75
76 typedef struct {
77     float *y;
78     const float *x;
79     float a;
80     size_t begin;
81     size_t end;
82     int tid;
83     int pin; // 1 = pin threads if linux
84 } task_t;
85
86 static void *worker(void *arg) {
87     task_t *t = (task_t*)arg;
88
89     #ifdef __linux__
90     if (t->pin) {
91         cpu_set_t set;
92         CPU_ZERO(&set);
93         CPU_SET(t->tid, &set);
94         pthread_setaffinity_np(pthread_self(), sizeof(set), &set);
95     }
96     #endif
97
98     axpy_simd_unroll4_chunk(t->y, t->x, t->a, t->begin, t->end);
99     return NULL;
100 }
101
102 int main(int argc, char **argv) {
103     size_t N = (argc > 1) ? (size_t)atoll(argv[1]) : (size_t)16 * 1024
        * 1024;
104     int R = (argc > 2) ? atoi(argv[2]) : 10;
105     float a = (argc > 3) ? (float)atof(argv[3]) : 1.2345f;
106     int T = (argc > 4) ? atoi(argv[4]) : 4;
107     int pin = (argc > 5) ? atoi(argv[5]) : 0;
108

```



```

109 float *x = (float*)malloc(N * sizeof(float));
110 float *y = (float*)malloc(N * sizeof(float));
111 if (!x || !y) { fprintf(stderr, "alloc failed\n"); return 1; }
112
113 init_arrays(x, y, N);
114
115 // Warmup single-thread chunk to prime pages/cache
116 axpy_simd_unroll4_chunk(y, x, a, 0, N);
117
118 pthread_t *threads = (pthread_t*)malloc((size_t)T * sizeof(
    pthread_t));
119 task_t *tasks = (task_t*)malloc((size_t)T * sizeof(task_t));
120 if (!threads || !tasks) return 1;
121
122 double t0 = now_sec();
123 for (int r = 0; r < R; r++) {
124     size_t chunk = N / (size_t)T;
125     size_t start = 0;
126     for (int t = 0; t < T; t++) {
127         size_t end = (t == T - 1) ? N : (start + chunk);
128
129         // Make chunk boundaries nicer for SIMD (optional hygiene):
130         // round end down to multiple of 8, except last thread.
131         if (t != T - 1) end &= ~(size_t)7;
132
133         tasks[t] = (task_t){ .y=y, .x=x, .a=a, .begin=start, .end=
            end, .tid=t, .pin=pin };
134         pthread_create(&threads[t], NULL, worker, &tasks[t]);
135         start = end;
136     }
137     for (int t = 0; t < T; t++) pthread_join(threads[t], NULL);
138 }
139 double t1 = now_sec();
140
141 double sec = t1 - t0;
142 double bytes = 12.0 * (double)N * (double)R;
143 double gbps = (bytes / sec) / 1e9;
144
145 printf("Variant 7 (Pthreads+SIMD) N=%zu R=%d T=%d pin=%d time=%.6
    f s BW~%.2f GB/s checksum=%f\n",
146     N, R, T, pin, sec, gbps, checksum(y, N));
147
148 free(threads); free(tasks);
149 free(x); free(y);
150 return 0;
151 }

```